

实验 5.2.2 进程页表：麒麟 V11 实验流程

1. 实验环境

远程虚拟机环境：

```
主机名：cs2-host1  
系统内核：Linux 6.6.0-32.12.v2505.ky11.x86_64  
页大小：4096 字节
```

教材原方案使用 `crash + kernel-debuginfo` 查看 `task_struct->mm->pgd`，再读取物理内存验证字符串内容。麒麟 V11 当前软件源无法提供已运行内核对应的 `kernel-debuginfo`，因此本实验改用内核模块直接调用内核页表遍历接口：

```
task_struct -> mm_struct -> pgd  
PGD -> P4D -> PUD -> PMD -> PTE -> PFN -> 物理地址
```

这种方式不依赖 `kernel-debuginfo`，但仍然完整展示了多级页表转换过程。

2. 准备开发环境

确认内核版本：

```
uname -r
```

安装与当前内核精确匹配的开发包（虚拟机已安装，仅作流程演示）：

```
sudo dnf -y install kernel-devel-$(uname -r)
```

本次安装结果包含：

```
kernel-devel-6.6.0-32.12.v2505.ky11.x86_64  
elfutils-libelf-devel  
zlib-devel  
zstd-devel
```

确认模块编译目录存在：

```
ls -ld /lib/modules/$(uname -r)/build
```

3. 实验代码

本地和远程均使用目录，请在自己的用户目录下创建该文件夹：

```
exp522_kylin/
```

主要文件：

show_virt_addr.c	目标用户程序，打印字符串虚拟地址
walk_pt.c	内核模块，按 PID 和虚拟地址遍历页表
Makefile	编译目标程序和内核模块

show_virt_addr.c的内容为：

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

int main(void)
{
    const char *str = "Hello world!\n";

    printf("pid=%d\n", getpid());
    printf("%s @ %p\n", str, (const void *)str);
    fflush(stdout);
    if (getchar() == EOF)
        pause();
    return 0;
}
```

walk_pt.c的内容为：

```
#include <linux/highmem.h>
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/mm.h>
#include <linux/module.h>
#include <linux/pid.h>
#include <linux/sched/mm.h>
#include <linux/sched/task.h>
#include <linux/types.h>
#include <asm/io.h>
#include <asm/pgtable.h>
```

```

/*
 * 实验 5.2.2：给定一个进程 PID 和该进程中的一个用户虚拟地址，
 * 从 task_struct 找到 mm_struct，再沿着进程页表逐级走到 PTE，
 * 最后把 PTE 中的 PFN 和页内偏移合成物理地址。
 *
 * 说明：这是教学实验代码，重点是展示地址转换过程，不是通用内核
 * 调试工具。实验时目标进程要保持运行，否则它的 mm_struct 和页表
 * 可能已经被释放。
 */

/* insmod 时传入的模块参数：pid=目标进程号，vaddr=目标用户虚拟地址。 */
static int pid = -1;
static unsigned long vaddr;

module_param(pid, int, 0444);
MODULE_PARM_DESC(pid, "Target process PID");
module_param(vaddr, ulong, 0444);
MODULE_PARM_DESC(vaddr, "Target user virtual address");

static void print_bytes(const unsigned char *buf, size_t len)
{
    /* 同时打印十六进制和可见 ASCII，便于确认物理地址处的内容。 */
    char hex[3 * 32 + 1];
    char ascii[33];
    size_t i;

    len = min_t(size_t, len, 32);
    for (i = 0; i < len; i++) {
        static const char digits[] = "0123456789abcdef";

        hex[i * 3] = digits[buf[i] >> 4];
        hex[i * 3 + 1] = digits[buf[i] & 0xf];
        hex[i * 3 + 2] = ' ';
        ascii[i] = (buf[i] >= 32 && buf[i] <= 126) ? buf[i] : '.';
    }
    hex[len * 3] = '\\0';
    ascii[len] = '\\0';

    pr_info("exp522: bytes(hex)=%s\\n", hex);
    pr_info("exp522: bytes(ascii)=\\\"%s\\\"\\n", ascii);
}

static int read_final_page(unsigned long pfn, unsigned long offset)
{
    struct page *page;
    void *mapped;
    unsigned char buf[32];
    size_t n;

    if (!pfn_valid(pfn)) {
        pr_err("exp522: final PFN 0x%x is not valid RAM\\n", pfn);
        return -EINVAL;
    }
}

```

```

/*
 * PTE 中保存的是 PFN，即物理页帧号。内核不能直接用 PFN 当指针，
 * 需要先转换成 struct page，再临时映射到内核虚拟地址空间读取。
 */
n = min_t(size_t, sizeof(buf), PAGE_SIZE - offset);
page = pfn_to_page(pfn);
mapped = kmap_local_page(page);
memcpy(buf, (unsigned char *)mapped + offset, n);
kunmap_local(mapped);

print_bytes(buf, n);
return 0;
}

static int walk_one_address(struct mm_struct *mm, unsigned long addr)
{
    /* x86-64 新内核通常是 5 级页表接口：PGD/P4D/PUD/PMD/PTE。 */
    pgd_t *pgd;
    p4d_t *p4d;
    pud_t *pud;
    pmd_t *pmd;
    pte_t *pte;
    /* 页内偏移就是虚拟地址低 12 位，4KB 页大小下范围是 0x000-0xfff。 */
    unsigned long offset = addr & ~PAGE_MASK;
    unsigned long pfn;
    unsigned long phys;
    int ret = 0;

    pr_info("exp522: vaddr=0x%lx page_offset=0x%lx PAGE_SIZE=%lu\n",
            addr, offset, PAGE_SIZE);
    pr_info("exp522: indexes pgd=%lu p4d=%lu pud=%lu pmd=%lu pte=%lu\n",
            pgd_index(addr), p4d_index(addr), pud_index(addr),
            pmd_index(addr), pte_index(addr));

    /*
     * 页表属于进程的内存描述符 mm_struct。遍历期间持有 mmap 读锁，
     * 防止用户态地址空间结构被并发修改。
     */
    mmap_read_lock(mm);

    /*
     * 第一级：PGD。pgd_offset(mm, addr) 根据 mm->pgd 和虚拟地址中的
     * PGD 索引，得到目标 PGD 项的内核虚拟地址。
     */
    pgd = pgd_offset(mm, addr);
    pr_info("exp522: PGD entry_addr=%px entry_phys=%pa value=0x%016llx\n",
            pgd, &(phys_addr_t){ virt_to_phys(pgd) },
            (unsigned long long)pgd_val(*pgd));
    if (pgd_none(*pgd) || pgd_bad(*pgd)) {
        pr_err("exp522: PGD is none/bad\n");
        ret = -EFAULT;
        goto out_unlock_mmap;
    }
}

```

```

/*
 * 第二级：P4D。若内核没有真正启用 5 级页表，P4D 可能被折叠，
 * 此时它和 PGD 看起来会指向同一项；使用统一接口即可。
 */
p4d = p4d_offset(pgd, addr);
pr_info("exp522: P4D entry_addr=%px entry_phys=%pa value=0x%016llx\n",
        p4d, &(phys_addr_t){ virt_to_phys(p4d) },
        (unsigned long long)p4d_val(*p4d));
if (p4d_none(*p4d) || p4d_bad(*p4d)) {
    pr_err("exp522: P4D is none/bad\n");
    ret = -EFAULT;
    goto out_unlock_mmap;
}
if (p4d_leaf(*p4d)) {
    pr_err("exp522: P4D leaf mapping is not handled\n");
    ret = -EOPNOTSUPP;
    goto out_unlock_mmap;
}

/* 第三级：PUD。普通 4KB 页映射会继续走到 PMD；大页可能在这里结束。 */
pud = pud_offset(p4d, addr);
pr_info("exp522: PUD entry_addr=%px entry_phys=%pa value=0x%016llx\n",
        pud, &(phys_addr_t){ virt_to_phys(pud) },
        (unsigned long long)pud_val(*pud));
if (pud_none(*pud) || pud_bad(*pud)) {
    pr_err("exp522: PUD is none/bad\n");
    ret = -EFAULT;
    goto out_unlock_mmap;
}
if (pud_leaf(*pud)) {
    /* 1GB 大页：PUD 直接指向大页，需要把大页内页号加到 PFN 上。 */
    pfn = pud_pfn(*pud) + ((addr & ~PUD_MASK) >> PAGE_SHIFT);
    phys = (pfn << PAGE_SHIFT) + offset;
    pr_info("exp522: PUD huge leaf pfn=0x%lx phys=0x%lx\n", pfn, phys);
    ret = read_final_page(pfn, offset);
    goto out_unlock_mmap;
}

/* 第四级：PMD。普通页继续走 PTE；透明大页常在 PMD 级结束。 */
pmd = pmd_offset(pud, addr);
pr_info("exp522: PMD entry_addr=%px entry_phys=%pa value=0x%016llx\n",
        pmd, &(phys_addr_t){ virt_to_phys(pmd) },
        (unsigned long long)pmd_val(*pmd));
if (pmd_none(*pmd) || pmd_bad(*pmd)) {
    pr_err("exp522: PMD is none/bad\n");
    ret = -EFAULT;
    goto out_unlock_mmap;
}
if (pmd_leaf(*pmd)) {
    /* 2MB 大页：PMD 直接指向大页，同样需要补上大页内页号。 */
    pfn = pmd_pfn(*pmd) + ((addr & ~PMD_MASK) >> PAGE_SHIFT);
    phys = (pfn << PAGE_SHIFT) + offset;
    pr_info("exp522: PMD huge leaf pfn=0x%lx phys=0x%lx\n", pfn, phys);
    ret = read_final_page(pfn, offset);
}

```

```

        goto out_unlock_mmap;
    }

    /*
     * 第五级：PTE。PTE 是普通 4KB 页映射的最后一级，里面保存最终
     * 物理页帧号 PFN 以及 present/user/rw 等标志位。
     */
    pte = pte_offset_kernel(pmd, addr);
    if (!pte) {
        pr_err("exp522: failed to map PTE\n");
        ret = -EFAULT;
        goto out_unlock_mmap;
    }

    pr_info("exp522: PTE entry_addr=%px entry_phys=%pa value=0x%016llx\n",
            pte, &(phys_addr_t){ virt_to_phys(pte) },
            (unsigned long long)pte_val(*pte));
    if (!pte_present(*pte)) {
        pr_err("exp522: PTE is not present\n");
        ret = -EFAULT;
        goto out_unlock_mmap;
    }

    /* pte_pfn() 去掉标志位，只取出页表项中的物理页帧号。 */
    pfn = pte_pfn(*pte);
    phys = (pfn << PAGE_SHIFT) + offset;
    pr_info("exp522: final pfn=0x%lx phys=0x%lx\n", pfn, phys);
    ret = read_final_page(pfn, offset);

out_unlock_mmap:
    mmap_read_unlock(mm);
    return ret;
}

static int __init walk_pt_init(void)
{
    struct pid *pid_struct;
    struct task_struct *task;
    struct mm_struct *mm;
    char comm[TASK_COMM_LEN];
    int ret;

    if (pid <= 0 || !vaddr) {
        pr_err("exp522: usage: insmod walk_pt.ko pid=<pid> vaddr=<hex-address>\n");
        return -EINVAL;
    }

    /* PID 号先转换为内核的 struct pid，再取到对应的 task_struct。 */
    pid_struct = find_get_pid(pid);
    if (!pid_struct) {
        pr_err("exp522: PID %d not found\n", pid);
        return -ESRCH;
    }

```

```

task = get_pid_task(pid_struct, PIDTYPE_PID);
put_pid(pid_struct);
if (!task) {
    pr_err("exp522: task for PID %d not found\n", pid);
    return -ESRCH;
}

/*
 * task_struct 是进程控制块。用户进程的地址空间由 mm_struct 描述，
 * 其中 mm->pgd 就是该进程顶级页表的内核虚拟地址。
 */
mm = get_task_mm(task);
get_task_comm(comm, task);
put_task_struct(task);
if (!mm) {
    pr_err("exp522: PID %d has no userspace mm\n", pid);
    return -EINVAL;
}

pr_info("exp522: target pid=%d comm=%s mm=%px pgd=%px pgd_phys=%pa\n",
        pid, comm, mm, mm->pgd,
        &(phys_addr_t){ virt_to_phys(mm->pgd) });

/* 找到 mm->pgd 后，开始对传入的用户虚拟地址做逐级页表转换。 */
ret = walk_one_address(mm, vaddr);
mmapput(mm);
return ret;
}

static void __exit walk_pt_exit(void)
{
    pr_info("exp522: module unloaded\n");
}

module_init(walk_pt_init);
module_exit(walk_pt_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Codex");
MODULE_DESCRIPTION("Experiment 5.2.2: walk a process page table for one
virtual address");

```

Makefile的内容为:

```

obj-m += walk_pt.o

KDIR ?= /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)

all: walk_pt.ko show_virt_addr

```

```
walk_pt.ko: walk_pt.c
$(MAKE) -C $(KDIR) M=$(PWD) modules

show_virt_addr: show_virt_addr.c
gcc -Wall -Wextra -O2 -no-pie -fno-pie -o show_virt_addr
show_virt_addr.c

clean:
$(MAKE) -C $(KDIR) M=$(PWD) clean
rm -f show_virt_addr
```

目标程序打印 "Hello world!\n" 所在虚拟地址，然后停住等待，保证页表仍然有效。内核模块参数为：

```
pid=<目标进程 PID>
vaddr=<目标虚拟地址>
```

模块内部依次读取：

```
pgd_offset()
p4d_offset()
pud_offset()
pmd_offset()
pte_offset_kernel()
pte_pfn()
```

最后计算：

$$\text{物理地址} = \text{PFN} * \text{PAGE_SIZE} + \text{页内偏移}$$

并从最终物理页帧中读出字节，验证是否为 "Hello world!"。

4. 编译

进入实验目录：

```
cd /home/stu-<学号>/exp522_kylin
make clean
make
```

目标程序需要编译为非 PIE 可执行文件，便于观察类似教材中的低地址用户虚拟地址：

```
gcc -Wall -Wextra -O2 -no-pie -fno-pie -o show_virt_addr show_virt_addr.c
```


确认 ELF 类型：

```
readelf -h show_virt_addr
```

本次结果：

类型：EXEC（可执行文件）
入口点地址：0x4010e0

5. 运行目标程序

远程非交互环境下，使用后台方式启动：

```
cd /home/stu-<学号>/exp522_kylin  
nohup ./show_virt_addr </dev/null > show.out 2>&1 &  
PID=$!  
cat show.out
```

本次目标程序输出：

```
pid=1552991  
Hello world!  
@ 0x40200c
```

因此：

目标 PID：1552991
字符串虚拟地址：0x40200c

6. 加载模块并遍历页表

执行：

```
sudo insmod ./walk_pt.ko pid=1552991 vaddr=0x40200c  
sudo dmesg | grep exp522 | tail -40  
sudo rmmod walk_pt
```

本次核心输出：

```
exp522: target pid=1552991 comm=show_virt_addr mm=ffff9402c4e23980
pgd=ffff9402dc02e000 pgd_phys=0x0000000011c02e000
exp522: vaddr=0x40200c page_offset=0xc PAGE_SIZE=4096
exp522: indexes pgd=0 p4d=0 pud=0 pmd=2 pte=2
exp522: PGD entry_addr=ffff9402dc02e000 entry_phys=0x0000000011c02e000
value=0x000000001f6db8067
exp522: P4D entry_addr=ffff9402dc02e000 entry_phys=0x0000000011c02e000
value=0x000000001f6db8067
exp522: PUD entry_addr=ffff9403b6db8000 entry_phys=0x000000001f6db8000
value=0x00000000172f96067
exp522: PMD entry_addr=ffff940332f96010 entry_phys=0x00000000172f96010
value=0x0000000013dd56067
exp522: PTE entry_addr=ffff9402fdd56010 entry_phys=0x0000000013dd56010
value=0x80000000122a52025
exp522: final pfn=0x122a52 phys=0x122a5200c
exp522: bytes(hex)=48 65 6c 6c 6f 20 77 6f 72 6c 64 21 0a 00 25 73 20 40 20
25 70 0a 00 00 01 1b 03 3b 28 00 00 00
exp522: bytes(ascii)="Hello world!..%s @ %p.....;(..."
```

可以看到最终物理地址处的字节以：

```
Hello world!
```

开头，与目标程序打印的字符串一致。

7. 地址转换计算

目标虚拟地址：

```
0x40200c
```

页内偏移：

```
0x40200c & 0xffff = 0x00c
```

模块计算出的各级索引：

```
PGD index = 0
P4D index = 0
PUD index = 0
PMD index = 2
PTE index = 2
offset    = 0xc
```

最终 PTE :

```
PTE value = 0x8000000122a52025
PFN       = 0x122a52
```

物理地址 :

```
PFN << 12 + offset
= 0x122a52 * 0x1000 + 0xc
= 0x122a5200c
```

该地址读出的内容 :

```
48 65 6c 6c 6f 20 77 6f 72 6c 64 21 0a
H e l l o   w o r l d ! \n
```

验证了虚拟地址 `0x40200c` 经进程页表转换后,指向的物理内容确实是目标字符串。

8. 使用 pagemap 交叉验证

作为独立核对,也读取 `/proc/<pid>/pagemap` :

```
PID=1552991
VADDR=0x40200c
OFF=$(( (VADDR / 4096) * 8 ))
ENTRY=$(sudo od -An -t x8 -j $OFF -N 8 /proc/$PID/pagemap | tr -d " ")
PFN=$(( 0x$ENTRY & 0x7fffffffffffffff ))
PHYS=$(( PFN * 4096 + (VADDR & 4095) ))
printf "pagemap_entry=0x%s\npagemap_pfn=0x%x\npagemap_phys=0x%x\n" "$ENTRY"
"$PFN" "$PHYS"
```

输出 :

```
pagemap_entry=0xa180000000122a52
pagemap_pfn=0x122a52
pagemap_phys=0x122a5200c
```

`pagemap` 得到的 PFN 和物理地址与内核模块输出完全一致。

9. 清理

卸载模块 :

```
sudo rmmod walk_pt
```

结束目标进程：

```
kill 1552991
```

如需清理编译产物：

```
make clean
```

10. 结论

在麒麟 V11 上，由于无法获得当前内核精确匹配的 `kernel-debuginfo`，教材中的 `crash` 路线不可行。本实验改用内核模块直接读取目标进程的页表，完成了从用户虚拟地址到物理地址的逐级转换：

```
0x40200c -> PGD/P4D/PUD/PMD/PTE -> PFN 0x122a52 -> 物理地址 0x122a5200c
```

最终从该物理地址读出的内容为 `"Hello world!\n"`，实验目标达成。